សប្តាហ៍ទី 10 · ផ្ទៀងផ្ទាត់ ប្រគល់ និងបង្ហាញ

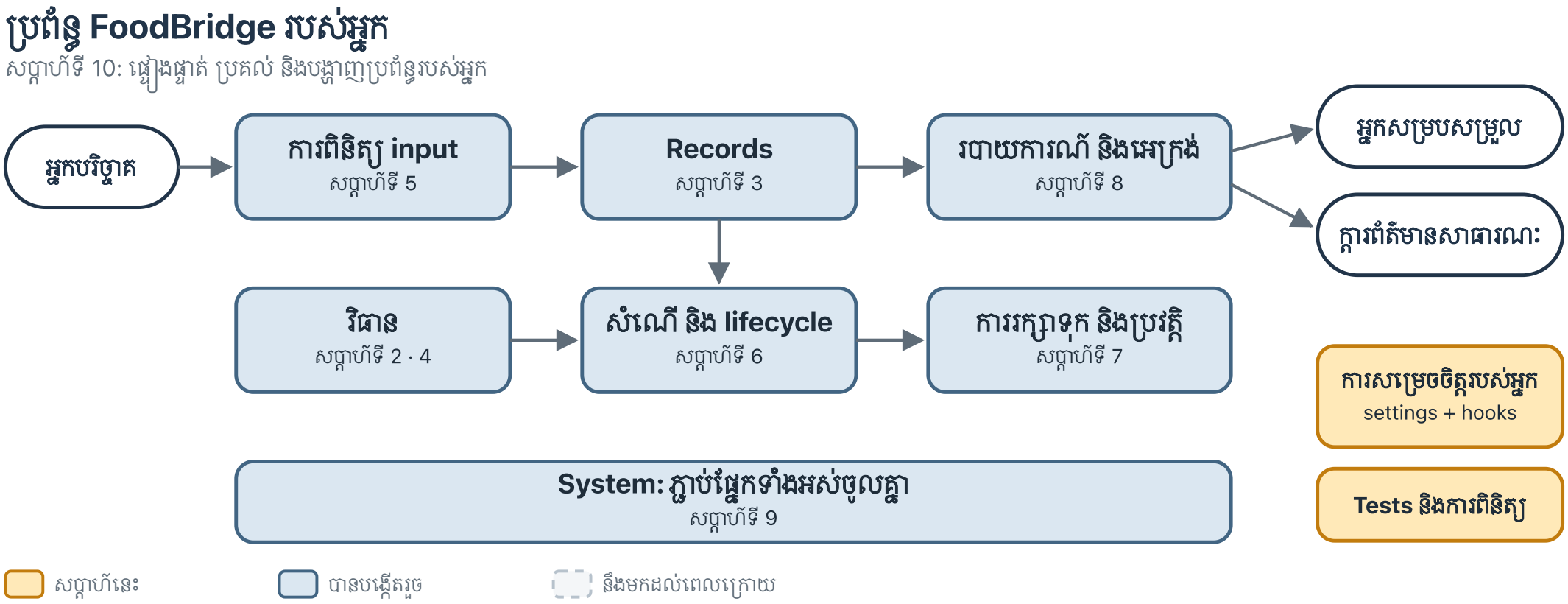
Programming for Information Systems

តើអ្នកដឹងយ៉ាងដូចម្តេចថា វាដំណើរការ ហើយតើអ្នកផ្សេងអាច run វាបាន ដោយគ្មានអ្នកដែរឬទេ?

សប្តាហ៍នេះ

សំណួរ	តើអ្នកដឹងយ៉ាងដូចម្តេចថា ប្រព័ន្ធរបស់អ្នកដំណើរការ ហើយតើអ្នកផ្សេងអាច run វាបាន ដោយគ្មានអ្នកដែរឬទេ?
Python	test function, ការអាន output របស់ test, clean run, ការពិនិត្យ code
គោលការណ៍ទី 10	ផ្ទៀងផ្ទាត់ មុននឹងទុកចិត្ត
អ្នកបង្កើត	test សម្រាប់ការសម្រេចចិត្តផ្ទាល់ខ្លួនរបស់អ្នក កំណត់ចំណាំប្រគល់ការងារ និង Final ដែលបានបញ្ចប់

ប្រព័ន្ធរបស់អ្នក ត្រឹមសប្តាហ៍ទី 10



គំនិតសំខាន់: ប្រព័ន្ធដែលអ្នកផ្សែងអាច run បាន

- ម្ចាស់ហាងម្នាក់ទៅឆ្ងាយ។ មិត្តម្នាក់មើលហាង ដោយប្រើ **កំណត់ចំណាំ** ខ្លីមួយ។
- មិត្តនោះសាកផ្ងើនៅព្រឹកដំបូង ខណៈដែលម្ចាស់ហាងនៅតែអាចទាក់ទងបាន។
- ក្នុងប្រព័ន្ធព័ត៌មាន នេះគឺជា **ការប្រគល់ការងារ** (handoff)។

ប្រព័ន្ធរបស់អ្នក **បានផ្ទៀងផ្ទាត់** ពេល: **1** test ជាប់ · **2** clean run ដំណើរការ · **3** មិត្តរួមថ្នាក់អាច run វាបាន ដោយប្រើតែការណែនាំរបស់អ្នក។

ការធ្វើឯកជន

test, output, clean run និងការពិនិត្យ code

test មួយ ពិនិត្យការសន្យាមួយ

```
def discount(price):
    if price >= 100:
        return 10
    return 0

def test_discount_at_the_boundary():
    assert discount(100) == 10, "exactly 100 gets the discount"
```

- ឈ្មោះចាប់ផ្តើមដោយ `test_` ហើយ **ប្រាប់ពីការសន្យា**។
- ជាប់ → មិន print អ្វីទេ។ ធ្លាក់ → `AssertionError` ជាមួយសាររបស់អ្នក។

Run test ច្រើន ដោយមិនឈប់

```
def run_tests(tests):
    passed = 0
    for test in tests:
        try:
            test()
            print("✅", test.__name__)
            passed = passed + 1
        except Exception as error:
            print("❌", test.__name__, "→", error)
    print(passed, "of", len(tests), "tests passed")
```

ការអាន output របស់ test

```
✅ test_public_board_hides_contacts
❌ test_25_portions_need_a_team → AssertionError:
   25 portions is my team minimum, so it needs a team
2 of 3 tests passed
```

1. **បន្ទាត់ចុងក្រោយ:** តើធ្លាក់ប៉ុន្មាន?
2. **ឈ្មោះ:** តើការសន្យាណាដែលខូច?
3. **សារ:** តើអ្វីគួរតែពិត?
4. **សម្រេចចិត្ត:** តើ code ខុស test ខុស ឬការសម្រេចចិត្តមួយបានប្តូរ?

clean run: Runtime → Restart session and run all

- notebook ចងចាំ cell ដែលអ្នកបាន **កែ ឬលុប**។
- វាអាចមើលទៅល្អសម្រាប់អ្នក ប៉ុន្តែ error សម្រាប់អ្នកផ្សេងទាំងអស់។
- **Colab:** Runtime → Restart session and run all។
- គ្រប់ cell run ពីលើចុះក្រោម ដោយគ្មាន error។ បន្ទាប់មក អាន output ម្តងទៀត។

```
total_today = 48      # វាយក្នុង cell មួយ ដែលក្រោយមកត្រូវបានលុប
# process ថ្មី: NameError: name 'total_today' is not defined
```

គោលការណ៍ទាំង 10 ជាបញ្ជីត្រួតពិនិត្យ

#	គោលការណ៍	សួរ
1	ឈ្មោះ និងឯកតាច្បាស់លាស់	តើឈ្មោះនីមួយៗប្រាប់ថា វាមានន័យអ្វី និងជាឯកតាអ្វី?
2	វិធានច្បាស់លាស់	លេខមានឈ្មោះ? គ្រប់ករណី? boundary បាន test?
3	record មួយ សម្រាប់របស់ពិតមួយ	តើការពិតនីមួយៗ រក្សាទុកតែម្តង ជាមួយ ID មួយ?
4	function មួយ ការងារមួយ	តើវា return ជំនួសឱ្យ print?
5	កុំទុកចិត្ត input	input ខុសត្រូវបានបដិសេធ ពន្យល់ ហើយគ្មានអ្វីប្រែប្រួល?
6	ស្ថានភាពខុស កើតឡើងពិបាក	តើវាអាចរំលងការអនុម័ត ឬថយក្រោយបានទេ?
7	កុំឱ្យបាត់បង់ទិន្នន័យ	តើមានអ្វីអាចត្រូវសរសេរជាន់ ឬបាត់បង់?
8	ព័ត៌មានត្រូវ មនុស្សត្រូវ	ទិន្នន័យឯកជនមានសុវត្ថិភាព? លេខស្មោះត្រង់?
9	បំបែកអ្វីដែលប្រែប្រួល	តើមានការសម្រេចចិត្ត កប់ក្នុងគ្រឿងម៉ាស៊ីនទេ?
10	ផ្ទៀងផ្ទាត់ មុនទុកចិត្ត	មាន test ទេ? បាន run ពីការចាប់ផ្តើមស្អាតទេ?

ការពិនិត្យ code ដែល AI សរសេរ

1. **អានការស្នើសុំ។** តើ code គួរសន្យាអ្វី?
2. **អាន code** ជាមួយបញ្ជីត្រួតពិនិត្យ។ កត់បញ្ហា និងបន្ទាត់។
3. **test edge case:** ធម្មតា boundary ចម្លែក (ទទេ សូន្យ បាត់)។
4. **ភ្ជាប់វាចូល ហើយតាមដាន** ⚠️**។** fallback មានន័យថា code របស់អ្នក *មិន* ត្រូវបានប្រើ។
5. **សម្រេចចិត្ត:** ទទួលយក កែ ឬបដិសេធ ហើយសរសេរមូលហេតុ។

> "វា run បាន" មិនដូចគ្នានឹង "វាត្រឹមត្រូវ" ទេ។

⚖️ ល្អ ឬមានហានិភ័យ?

Version A (ពីជំនួយការ AI)

```
def ai_pickup_label(portions, approved):
    """Improved pickup label."""
    if portions >= 25:
        print("team")
    else:
        print("standard")
```

Version B (វិធានសប្តាហ៍ទី 2 របស់អ្នក)

```
def pickup_label(portions, approved):
    if not approved:
        return "hold"
    if portions >= TEAM_PICKUP_MIN_PORTIONS:
        return "team"
    return "standard"
```

"ខ្ញុំបាន test វា ហើយវាដំណើរការ។"

អ្វីដែលការពិនិត្យរកឃើញ

- **វា return None** → ប្រព័ន្ធត្រឡប់ទៅប្រើ default។ របាយការណ៍ *មើលទៅ* ត្រឹមត្រូវ ប៉ុន្តែ code របស់ AI **មិនដែលត្រូវបានប្រើ**។
- **មើលរំលងការអនុម័ត** → សំណើថ្មីនឹងបាន `team` មិនមែន `hold`។ អ្នកស្ម័គ្រចិត្តបើកទ្វានទៅយកអាហារ ដែលអាចនឹងត្រូវបដិសេធ។
- **លេខវេទមន្ត 25** → setting របស់អ្នកសម្របសម្រួលត្រូវបានមើលរំលង; អេក្រង់មិនស្របគ្នា។
- **គ្មាន test** → `bridge.check` បង្ហាញ ❌ ភ្លាមៗ។

> ចង់បានក្រុម ចាប់ពី 25? ប្តូរ **setting មួយ**។ មិនត្រូវការ code ថ្មីទេ។

អ្នកបង្កើត: test ដែលការពារការសម្រេចចិត្តរបស់អ្នក

```python
def test_offer_without_contact_is_refused():
    system = fresh_system()
    row = offer_row(system, 10)
    row["contact"] = "  "
    offer_id, errors = system.add_offer(row)
    assert offer_id is None, "contact is required in my system"
```

- test ពីរ សម្រាប់ការសម្រេចចិត្ត **មួយ**: boundary និងម្ខាងទៀត។
- អានលេខពី `system.settings` មិនមែនពីការចងចាំទេ។

Run ការផ្ទៀងផ្ទាត់ពេញលេញ

```
final_check = bridge.System(MY_SETTINGS, hooks=MY_HOOKS)
final_check.verify()
run_tests(example_tests + my_tests)
```

- **ការពិនិត្យ kit:** ត្រូវតែ ✅ ទាំងអស់។
- **ប្រព័ន្ធរបស់អ្នក:** ការសម្រេចចិត្ត និង hooks របស់អ្នក និង ⚠️ ណាមួយដែលត្រូវពន្យល់។
- **test របស់អ្នក:** ការសន្យារបស់អ្នក ✅ ទាំងអស់។

កំណត់ចំណាំប្រគល់ការងារ

អ្វីដែលប្រព័ន្ធរបស់ខ្ញុំធ្វើ	អ្នកណាប្រើវា សម្រាប់អ្វី (2 ប្រយោគ)
របៀប run វា	ជំហានដែលមានលេខរៀង និងអ្វីដែលអ្នកគួរឃើញ
ការសម្រេចចិត្តរបស់ខ្ញុំ	3 ដែលសំខាន់បំផុត: setting, value, ហេតុផល
អ្វីដែលខ្ញុំបាន test និងរបៀប	test, clean run
ដែនកំណត់ដែលដឹងស្រាប់	អ្វីដែលវាមិនធ្វើ; ⚠️ ណាមួយ

វគ្គទី 2 · ការបង្ហាញដោយ operator

អ្នកបង្កើតប្រព័ន្ធ	operator
ផ្តល់ notebook និងកំណត់ចំណាំប្រគល់ការងារ	បើកវានៅលើកុំព្យូទ័រ **ផ្ទាល់ខ្លួន**
មើល **ដោយស្ងៀមស្ងាត់**; មិនចង្អុល	ធ្វើតាម **តែ** កំណត់ចំណាំ ហើយនិយាយឮៗពីអ្វីដែលកំពុងគិត
កត់ត្រាកន្លែងដែល operator ស្ទាក់ស្ទើរ	បំពេញកំណត់ត្រា test ដោយមិត្តរួមថ្នាក់

- បន្ទាប់មក កែកំណត់ចំណាំ test ជំហាននោះម្តងទៀត ហើយប្តូរតួនាទីគ្នា។

កំណត់ត្រា test ដោយមិត្តរួមថ្នាក់

ជំហានក្នុងកំណត់ចំណាំ	អ្វីដែល operator បានធ្វើ	អ្វីដែលបានកើតឡើង	កំណត់សម្គាល់
1		ដំណើរការ / ត្រូវការជំនួយ / មិនដំណើរការ	
2			
3			

- ព្រមទាំង: ការលំបាកដំបូង · តើអ្នកបង្កើតប្រព័ន្ធបាននិយាយទេ? · លទ្ធផល `verify()` · ការផ្លាស់ប្តូរមួយដែលបានធ្វើ · បាន test ម្តងទៀតទេ?

បទបង្ហាញ 3 នាទី

ផ្នែក	ពេលវេលា
1 · មនុស្ស និងបញ្ហា	0:30
2 · ការសម្រេចចិត្តរបស់ខ្ញុំ (value ជម្រើសផ្សេង អ្នកណារងផលប៉ះពាល់)	0:45
3 · Demo: output មួយ ដែលការសម្រេចចិត្តរបស់អ្នកបានប្តូរ	0:45
4 · អ្វីដែលខ្ញុំបាន test	0:30
5 · អ្វីដែលខ្ញុំនឹងកែលម្អបន្ទាប់	0:30

វគ្គទី 3 · ការផ្ទៀងផ្ទាត់ម្ភាក់ៗ

- អ្នកពន្យល់ **ការសម្រេចចិត្តមួយ** និង **test មួយ** ដល់គ្រូ។
- ត្រៀម run test របស់អ្នក និងប្រាប់ថា ការផ្លាស់ប្តូរអ្វីនឹងធ្វើឱ្យវាធ្លាក់។
- គ្រូអាចឱ្យ **សំណើដែលមិនធ្លាប់ឃើញ** មួយដល់អ្នក: ទាយជាមុនសិន រួច run។
- អ្នកផ្សេងទៀតបញ្ចប់ ហើយដាក់ស្នើ Final។

ដាក់ស្នើ Final project

- `Final_Project.ipynb`: **Runtime → Restart session and run all** គ្មាន error។
- `verify()` ✅ ទាំងអស់ · ការសម្រេចចិត្តយ៉ាងហោចណាស់ 3 ជាមួយកំណត់ត្រាការសម្រេចចិត្ត។
- ផ្នែកទី 5: test ការសម្រេចចិត្តទាំងពីររបស់អ្នក ✅ កំណត់ចំណាំប្រគល់ការងារ និងកំណត់ត្រា test ដោយមិត្តរួមថ្នាក់។
- ផ្ញើ email ទៅ **submit.progmis@gmail.com**។ ថ្ងៃផុតកំណត់ និងឈ្មោះ file នឹងប្រកាសក្នុងថ្នាក់។

អ្វីដែលអ្នកអាចធ្វើបាន កម្មវិធីនេះ

- អាន code, test វា និង **វិនិច្ឆ័យ** វា រួមទាំង code ដែល AI សរសេរឱ្យអ្នក។
- បន្តសួរសំណួរពីរ:

> *អ្នកណាឯងផលប៉ះពាល់? និង តើខ្ញុំដឹងយ៉ាងដូចម្តេចថា វាដំណើរការ?*